

TRACEDET: An Interactive Dashboard for Post-Training Analysis of Object Detection Models

Jaival Bhuptani*, Samprith Kalakata*, Abhipal Sharma*

*Tandon School of Engineering, New York University

jnb9582@nyu.edu srk9068@nyu.edu as20410@nyu.edu

I. INTRODUCTION

Understanding why an object detection model fails is critical for improving robustness and interpretability. Typically when a machine learning engineer or researcher trains a model, they receive training loss per epoch and a singular metric of model performance, typically mean average precision (mAP) in the case of object detection models.

Problem Statement: however, this singular aggregate metric abstracts away the complex, heterogeneous nature of detection failures. A stagnant mAP can mask critical underlying issues, such as a model trading localization precision for classification accuracy, or hallucinating background objects while missing difficult instances. Without dissecting these specific failure modes - what the model mistakes, where it looks, and how these behaviors shift during training - practitioners are left guessing whether to adjust hyperparameters, refine the architecture, or clean the dataset.

While individual tools exist for error decomposition (TIDE [1]) and visual explanation (Grad-CAM [2], feature maps), they are rarely unified into an integrated, interactive post-training analysis workflow. In this project, we developed TRACEDET, a dashboard that combines:

- TIDE-based quantitative error breakdown,
- Temporal error evolution across epochs,
- Layerwise Grad-CAM and feature-map visualizations, and
- Interactive inspection tools for failure cases

to enable a deeper understanding of model behavior after training.

We fine-tuned a pre-trained Faster R-CNN model on a subset of COCO 2017 to detect 2 classes (people and vehicles). We track errors (classification, localization, both [classification and localization], duplicate, background, and missing) across 10 epochs, generate feature maps and Grad-CAM heatmaps for all images at epochs 1, 5, and 10. The resulting system allows machine learning engineers and researchers to trace persistent failures and how they evolve over time, connect them to attention shifts, and uncover underlying error signatures that

impact mAP. You can find our code for both the dashboard and model training here: https://github.com/ksam98/viz_ml_final_project/tree/main

II. RESEARCH QUESTIONS

Our investigation focused on the following:

- What types of errors is the detector making? Are failures due to misclassification, poor localization, false positives, or duplicate predictions?
- How do these errors evolve across training epochs? Does the model correct certain error types while others persist?
- Can temporal Grad-CAM and feature maps reveal why those errors occur?
- Does the attention focus move toward the correct object regions as training progresses?
- Do specific TIDE error categories correlate with distinctive attention patterns? e.g.,
 - Localization errors → diffuse/shifted activations
 - Classification errors → attention on nondiscriminative background
- Can an interactive dashboard help practitioners identify failure modes more quickly?
- And can it guide model debugging or dataset improvements?

III. RELATED WORKS

Error analysis and interpretability in object detection have been extensively studied. Bolya et al. [1] introduced **TIDE**, which decomposes detection errors into categories such as classification, localization, duplicates, and background false positives, enabling fine-grained analysis across detectors like Faster R-CNN, YOLO, and DETR.

Visual interpretability methods, including **Grad-CAM** [2], **Grad-CAM++** [3], **Layer-CAM** [4], and **Score-CAM** [5], generate class-discriminative heatmaps to provide insights into model attention. However, these techniques typically focus on single snapshots and do not link attention maps with quantitative error decomposition over training.

Our framework addresses this gap by unifying **TIDE-based error analysis** with **layerwise and epochwise Grad-CAM visualizations** in an interactive dashboard. This allows users to explore how detection failures develop over training, inspect individual misclassifications, and gain both quantitative and qualitative insights into model behavior.

IV. SYSTEM OVERVIEW

Our system is composed of four main components:

1 Model Training & Data Generation

- Fine-tuned Faster R-CNN for 2 classes: People and Vehicles.
- Trained for 10 epochs on COCO-2017.
- Extracted:
 - * Predictions & ground truths
 - * Feature maps and Grad-CAM heatmaps at epochs 1, 5, 10 for all images
 - * TIDE error breakdown per epoch

2 Global Error Evolution (TIDE Dashboard)

- Computed TIDE scores across epochs:
 - * Classification error
 - * Localization error
 - * Both (classification and localization) error
 - * Background false positives
 - * Duplicate predictions
 - * Missed predictions
- Built interactive visualizations

3 Image-Grid Inspection

- For each error type, we display a grid of images with errors corresponding to that type, sorted from highest to lowest error count.
- Users can click on an image for image-level inspection.

4 Image-level Inspection

- For each failing example:
 - * Display model predictions
 - * Display bounding boxes demonstrating the error
 - * Show Grad-CAM overlays
 - * Show feature maps from the FPN module at different resolutions.
- This reveals how attention changes over time.

5 Temporal Analysis

- For selected images, compare Grad-CAM maps at:
 - * Epoch 1 → Epoch 5 → Epoch 10
- Used to identify whether attention “corrects” as training progresses.

A. User Flow

The user begins on the global TIDE error on the **Dashboard**, where they can explore how different error categories - such as **classification, localization, background false**

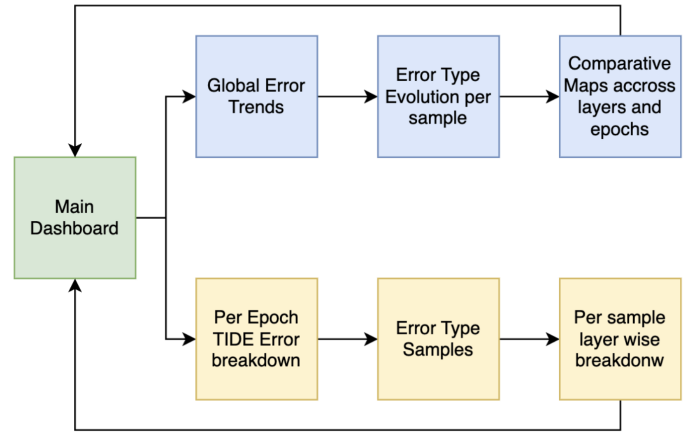


Fig. 1. User interaction flow for the visualization dashboard.

positives, and duplicates - evolve across epochs through interactive visualizations. After observing these global trends, the user selects a specific error type to focus on, which filters the dashboard to highlight only the images contributing to that error category. Choosing an epoch and error category then reveals an image grid of failure examples, with images ordered from highest to lowest error counts for that category, allowing the user to quickly identify problematic samples. Clicking on any image opens an in-depth inspection view, where the model’s predictions, ground-truth annotations, intermediate feature maps, and **layerwise Grad-CAM heatmaps** are displayed side by side. From here, the user can move through different layers or scroll across epochs to examine how the model’s attention shifts and whether the image’s failure mode resolves or persists over time. This seamless flow - from global statistics to fine-grained visual explanations - helps users understand not only what errors occurred but also why they happened and how they evolved throughout training.

B. Datasets

We finetuned a pretrained FasterRCNN model on:

- COCO-2017 validation subset
 - 2500 finetuning images
 - 564 validation images
 - Filtered to person and vehicle classes for focused analysis

COCO’s variation in occlusion, small objects, and crowded scenes makes it ideal for studying error sources like localization drift and false positives.

C. Implementation Stack

- Backend / Model Training: PyTorch, torchvision
- Error Metrics: TIDE

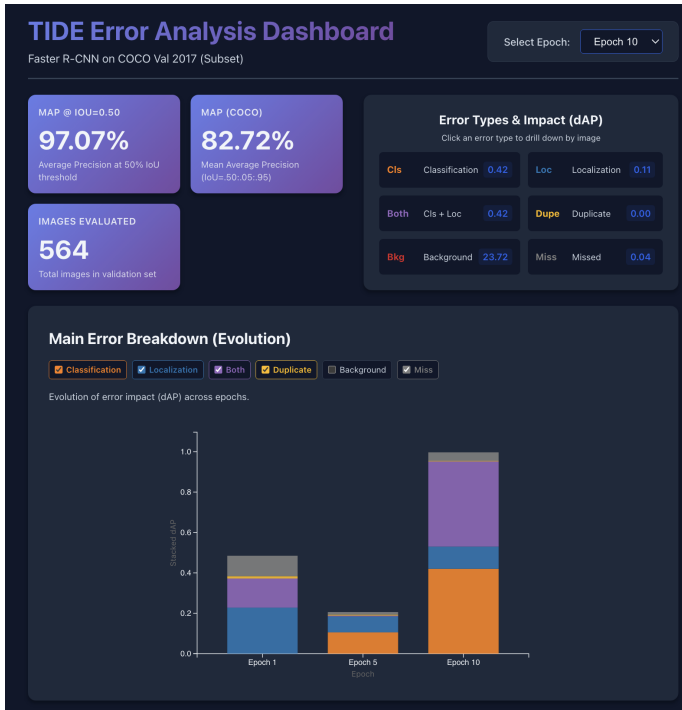


Fig. 2. Tide Dashboard

- Explainability: Grad-CAM, intermediate feature maps
- Dashboard: React + Vite + Plotly.js
- Visualization Modules (D3.js):
 - Parallel coordinates
 - 3D surface plots
 - Stacked area charts
 - Temporal Grad-CAM viewer

V. KEY FINDINGS

We conducted a small user study with five participants who have prior experience in computer vision. The goal was to evaluate whether our visualization system (TIDE dashboard, error-type image grids, FPN maps, and temporal Grad-CAM analysis) improved users' ability to understand Faster R-CNN's behavior across training epochs. The key findings are summarized below.

- 1) **Improved Error Diagnosis with TIDE Visualizations.** All participants reported that the global TIDE dashboard made it substantially easier to interpret how different error types (classification, localization, background false positives, duplicates, misses) evolved from epoch 1 to epoch 10.
- 2) **Image-Grid Sorting Helped Prioritize Failure Cases.** Participants noted that the ranked image grids for each error type enabled quick identification of "high-impact" failure cases. This structure was perceived as more efficient than browsing random samples and helped users form hypotheses about dataset-specific biases.

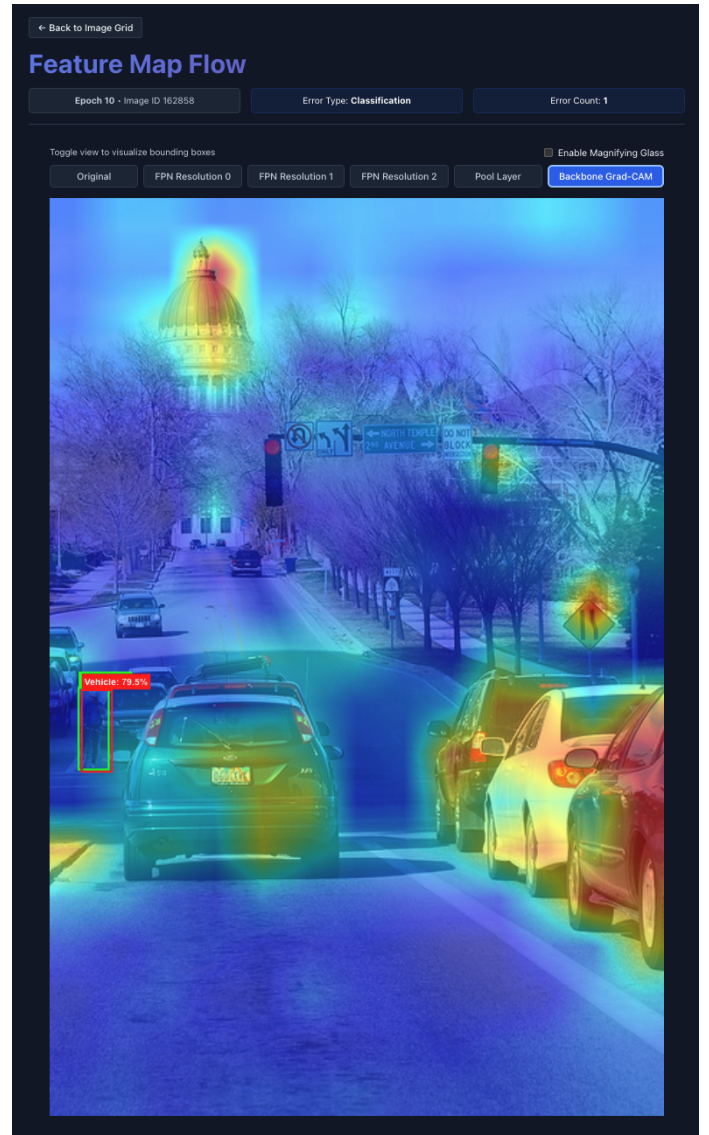


Fig. 3. Image level inspection (With Grad-CAM overlays)

- 3) **Grad-CAM and FPN Maps Enhanced Understanding, but Required Expertise.** All participants found Grad-CAM overlays useful for identifying where the detector focused incorrectly. The multi-scale FPN feature maps were considered informative by expert-level users, but two participants commented that interpreting four scales (P2–P5) introduced cognitive load.
- 4) **Temporal Analysis Made Model Progression Intuitive.** Participants consistently highlighted the temporal comparison (epoch 1 → epoch 5 → epoch 10) as one of the most helpful features. It allowed them to clearly see whether the model's attention became more localized, shifted to correct object regions, or remained unstable over training.
- 5) **Explanations Increased Trust When Attention Was Semantically Meaningful.** Users reported higher trust in the model when Grad-CAM heatmaps aligned with

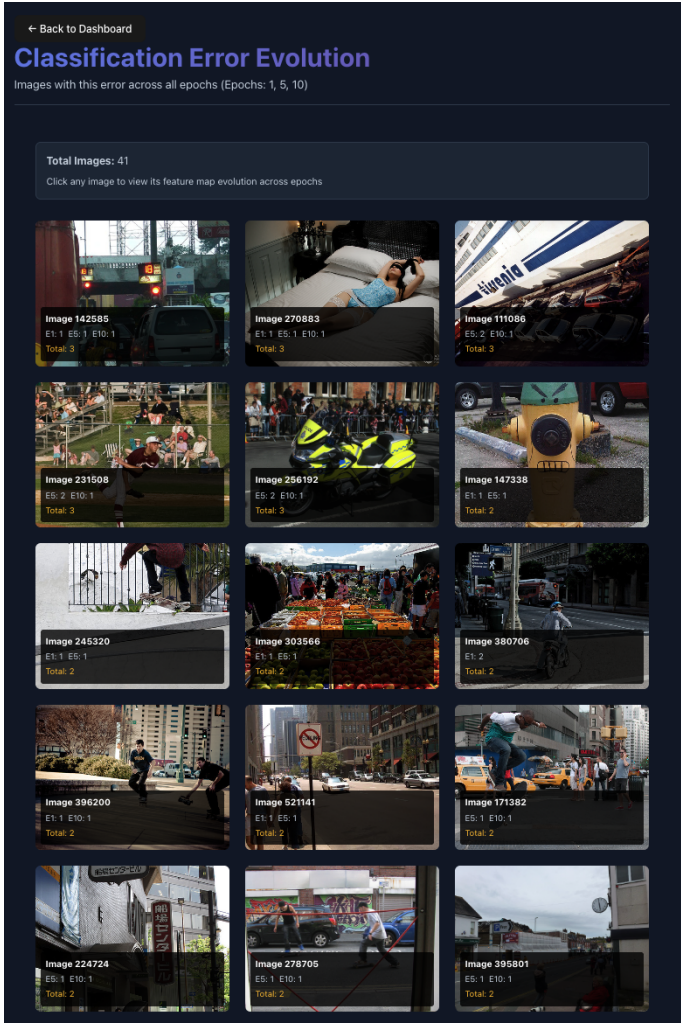


Fig. 4. Temporal Analysis Across Epochs

object regions and lower trust when heatmaps appeared diffuse or focused on background. This suggests that attention quality directly impacts perceived model reliability.

VI. FUTURE WORK

While TRACEDET currently serves as a powerful post-hoc diagnostic tool, bridging the gap between error identification and model correction remains a key frontier. Future work can focus on transforming the system from a passive analyzer into an active, real-time assistant that can intervene during the training lifecycle. The following several key expansions can enhance scalability, interpretability, and actionable utility:

- **Real-Time 'In-Training' Dashboard:** Integrating the dashboard directly into the training loop (e.g., via TensorBoard callbacks) to provide live alerts for error spikes. This would allow engineers to identify and abort di-

verging runs early, rather than waiting for post-training analysis.

- **Cross-Architecture Compatibility:** Extending support beyond Faster R-CNN to modern detectors like YOLOX, DETR, and DINO, enabling researchers to perform comparative error analysis across varying architectural paradigms.
- **Automated Failure Clustering:** utilizing unsupervised clustering on feature maps to automatically group visually similar failure modes (e.g., "dark objects" or "occlusions") without manual inspection.
- **Prescriptive Remediation:** implementing TIDE-driven feedback that suggests specific fixes, such as "increase NMS threshold" for duplicate errors or "add background augmentation" for false positives.
- **Advanced Explainability:** incorporating finer-grained attribution methods like Layer-CAM or Score-CAM to better disentangle object localization features from classification features.

REFERENCES

- [1] Bolya, D., et al., "TIDE: A General Toolbox for Identifying Object Detection Errors," ECCV, 2020.
- [2] Selvaraju, R. R., et al., "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization," ICCV, 2017.
- [3] Chattopadhyay, P., et al., "Grad-CAM++: Improved Visual Explanations for Deep Convolutional Networks," AAAI, 2018.
- [4] Jiang, M., et al., "Layer-CAM: Exploring Hierarchical Class Activation Maps for Deep CNNs," CVPR, 2021.
- [5] Wang, Y., et al., "Score-CAM: Score-Weighted Visual Explanations for CNNs," CVPR, 2020.